0x01 简介

大家好，我们是 NOP Team ，很久没和大家见面了，今天更新一篇关于 Linux 平台关于用户管理设计的技术文章

在之前学习 Linux 系统知识、使用 Linux 系统时，可能会了解用户、用户组、`/etc/passwd` 、`/etc/shadow` ，以及一些用户管理操作

在对 Linux 系统进行攻击测试的过程中，可能会关注当前获取的 `shell` 的用户权限以及如何提权

但是在此过程中 Linux 系统到底发生了什么？ 系统底层到底是如何处理多用户操作、用户权限变化等内容的呢？
这就是这篇文章的主要内容

0x02 什么是用户？

大家看到这标题可能会一愣，我用了这么久系统，你问我什么是用户？
那我们视角下的用户，从 Linux 系统内核的视角来说，用户是什么呢？

肯定是一串 01 组成的二进制数据，这是肯定的

这段二进制数据包含哪些内容呢？ 包括用户名？ 家目录？ 用户id？ Session ？ 所属组信息？
这些二进制数据要完成哪些任务呢？ 包括区分用户？ 权限变化？

这些问题，都得去 Linux 内核源代码中找答案了

0x03 内核根本不认识用户名

Linux 内核从头到尾只认数字。用户名是给人类看的，内核只认 UID（User ID）

但也不是说一个用户创建了，就会在内核某个内存空间内就保留它的 UID 等信息，其实一个用户的信息映入内核眼帘的契机是 —— 用户创建进程，例如用户登录启动 bash、用户执行 vim 等

每个创建的进程会在内核中对应一个 `task_struct` 结构体，其中在用户管理方面最关键的是 `real_cred` 和 `cred` 两个指针，直接指向一个 `cred` 结构体；内核还会保存 `ptracer_cred` 作为 `ptrace` 附加时的凭证快照。
权限判定的主体依据就是这些凭证对象：

```
// Linux kernel: include/linux/sched.h (简化)
struct task_struct {
    // ... 几百个字段 ...

    /* Tracer's credentials at attach: */
    const struct cred __rcu *ptracer_cred;  // 调试者的凭证快照 (ptrace 附加时记录)

    /* Objective and real subjective task credentials (COW): */
    const struct cred __rcu *real_cred;     // 真实凭证 ("你是谁")

    /* Effective (overridable) subjective task credentials (COW): */
    const struct cred __rcu *cred;          // 有效凭证 ("你现在以谁行事")

    // ...
};

// Linux kernel: include/linux/cred.h (节选，按当前主线源码字段名裁剪)
struct cred {
    atomic_long_t usage;      // 引用计数

    kuid_t uid;               // 真实 UID (Real UID)
```

```
    kgid_t gid;                // 真实 GID
    kuid_t suid;               // 保存的 UID (Saved UID)
    kgid_t sgid;               // 保存的 GID
    kuid_t euid;               // 有效 UID (Effective UID)
    kgid_t egid;               // 有效 GID
    kuid_t fsuid;              // 文件系统 UID (FS UID)
    kgid_t fsgid;              // 文件系统 GID

    unsigned securebits;       // securebits

    kernel_cap_t cap_inheritable;  // 可继承能力
    kernel_cap_t cap_permitted;    // 允许能力
    kernel_cap_t cap_effective;    // 生效能力
    kernel_cap_t cap_bset;         // bounding set
    kernel_cap_t cap_ambient;      // ambient set

    void *security;                // LSM 安全上下文 (如 SELinux/AppArmor)
    struct user_namespace *user_ns;
    struct group_info *group_info; // 补充组列表
};
```

这两个结构体都保存在内核的内存中，后续关于用户的相关操作也是通过它们的改变来完成的
想想也知道，如果保存在用户空间，用户不就可以自己自定义自己的权限了嘛，那就随意提权了，所以肯定存在于内核内存空间

通常情况下，进程只向内核提出需求，而不管什么用户权限之类的内容，都由内核来做判断。当然了，很多人写程序从用户友好等方面出发，会主动通过系统调用 (例如 getuid) 来获取当前用户的 uid 等信息。因此内核中有一个宏专门来满足这个频繁用到的需求

```
// 当前主线内核中的宏
#define current_cred() \
    rcu_dereference_protected(current->cred, 1)
// current 是宏，指向当前 CPU 正在执行的 task_struct
```

核心认知： `cred` 对象的地址直接存储在 `task_struct` 中，内核做权限检查时通过指针直接访问，主体是 O(1) 的内存访问（只做一次指针解引用）。这里不会去查用户名数据库；真正参与判定的是内存里的 UID/GID、组列表、Capabilities、LSM 上下文等数据。

到这里，大家可能已经清楚一件事：内核根本不知道你这系统上有多少用户，什么用户名，什么 UID ，只有在用户动起来了（登录、创建进程等）内核才会通过进程机制存储进程有关的用户的信息

cred 存储在哪里

上面说了 cred 存在于内核内存空间，具体来说，`struct cred` 对象通过内核的 **slab 分配器**分配：

```
// kernel/cred.c
static struct kmem_cache *cred_jar;

// 初始化时创建专用 slab 缓存（省略 SLAB_PANIC | SLAB_ACCOUNT 标志）
cred_jar = KMEM_CACHE(cred, SLAB_HWCACHE_ALIGN);

// 分配新 cred 时从 slab 拿（prepare_creds 用 kmem_cache_alloc，因为紧接 memcpy 覆盖全部字段）
struct cred *new = kmem_cache_alloc(cred_jar, GFP_KERNEL);
```

这个地址在内核虚拟地址空间中（如 `0xFFFF8880_...`），用户进程**永远无法直接访问**：

```
用户空间内存（0x0000_0000_0000 ~ 0x0000_7FFF_FFFF_FFFF）
  ← 进程的代码、堆、栈、malloc 分配的东西
  ← 用户程序可以直接读写
  ← cred 不在这里，进程碰不到

内核空间内存（0xFFFF_8000_0000_0000 ~ 0xFFFF_FFFF_FFFF_FFFF）
  ← 内核代码、内核数据结构、slab 分配器管理的对象
  ← cred 在这里分配，只有内核代码能访问
  ← 用户程序通过系统调用陷进来后，内核代码代为读取
```

用户空间的程序想"看到"自己的 uid，必须走系统调用，拿到的只是一个**数值副本**：

```
用户程序调用 getuid()
        ↓
syscall 陷入内核
        ↓
内核代码执行：
  return current->cred->uid;   // 内核读自己的内存
        ↓
返回值通过寄存器传回用户空间
        ↓
用户程序拿到 1000
```

用户程序从始至终都没有接触过 cred 对象本身。

0x04 cred 结构体解析

由于这篇文章不是聊进程的，所以关于我们只需要知道 `task_struct` 有两个指针指向 cred 对象的地址即可，这两个指针的差异对比如下：

	`real_cred`	`cred`
内核注释	Objective and real subjective task credentials (COW)	Effective (overridable) subjective task credentials (COW)
文档概括	真实凭证（"你是谁"）	有效凭证（"你现在以谁行事"）
作用	记录进程的真实身份；其他进程检查本进程身份时读这个	本进程执行操作时做权限判定读这个
常见状态	大多数时候与 `cred` 指向同一个 cred 对象	大多数时候与 `real_cred` 指向同一个 cred 对象
何时会不同	`override_creds()` 后保持不变	`override_creds()` 临时替换，`revert_creds()` 恢复
替换方式	在 `commit_creds()` 中替换	在 `commit_creds()` 中替换
获取方式	`current->real_cred`	`current->cred`，也通过 `current_cred()` 宏

`cred` 结构体的具体内容字段意义如下（后面会重点讨论一些字段）：

字段	类型	含义
`usage`	`atomic_long_t`	引用计数，跟踪有多少 task_struct（通过 real_cred/cred 指针）持有此 cred 对象
`uid`	`kuid_t`	真实 UID（Real UID），标识进程的真正身份
`gid`	`kgid_t`	真实 GID（Real GID），标识进程的真正组身份
`suid`	`kuid_t`	保存的 UID（Saved UID），保存之前的有效 UID，允许在许可范围内回切
`sgid`	`kgid_t`	保存的 GID（Saved GID），保存之前的有效 GID，允许在许可范围内回切
`euid`	`kuid_t`	有效 UID（Effective UID），权限判定的主要依据，决定"你现在以谁行事"
`egid`	`kgid_t`	有效 GID（Effective GID），权限判定中使用的有效组身份
`fsuid`	`kuid_t`	文件系统 UID，VFS 文件操作权限判定使用，通常跟随 euid 联动
`fsgid`	`kgid_t`	文件系统 GID，VFS 文件操作权限判定使用，通常跟随 egid 联动
`securebits`	`unsigned`	SUID-less 安全管理标志位，控制 capability 提升行为的细粒度开关
`cap_inheritable`	`kernel_cap_t`	可继承能力集，execve 时子进程可以继承的 capabilities
`cap_permitted`	`kernel_cap_t`	允许能力集，进程被允许持有的 capabilities 上限
`cap_effective`	`kernel_cap_t`	生效能力集，当前实际可使用的 capabilities，权限检查直接读取这个

`cap_bset`	`kernel_cap_t`	能力边界集（Bounding Set），限制 execve 后子进程可获得的 capabilities
`cap_ambient`	`kernel_cap_t`	环境能力集（Ambient Set），非 root 进程在 execve 时可保留的 capabilities
`jit_keyring`	`unsigned char`	默认密钥环，请求密钥时自动附加的目标密钥环类型（需 CONFIG_KEYS）
`session_keyring`	`struct key *`	会话密钥环，fork 时继承的密钥环（需 CONFIG_KEYS）
`process_keyring`	`struct key *`	进程密钥环，进程私有的密钥环（需 CONFIG_KEYS）
`thread_keyring`	`struct key *`	线程密钥环，线程私有的密钥环（需 CONFIG_KEYS）
`request_key_auth`	`struct key *`	委托的 request_key 授权令牌（需 CONFIG_KEYS）
`security`	`void *`	LSM 安全上下文指针，由 SELinux/AppArmor/Smack 等安全模块使用（需 CONFIG_SECURITY）
`user`	`struct user_struct *`	真实用户 ID 订阅，跟踪用户的系统资源使用（如进程数、文件数）
`user_ns`	`struct user_namespace *`	用户命名空间，此 cred 中 capabilities 和密钥环所相对的命名空间
`ucounts`	`struct ucounts *`	用户资源计数，跟踪用户在命名空间层级中的资源限制
`group_info`	`struct group_info *`	补充组列表，进程所属的所有附加组信息，权限判定时直接查询
`non_rcu`	`int`（union 成员）	标记是否可以跳过 RCU 删除流程
`rcu`	`struct rcu_head`（union 成员）	RCU 删除钩子，cred 对象通过 RCU 机制安全释放

接下来对主要的内容进行解析

1. 四种 UID 各自的角色

用一个具体场景理解：

```
用户 ubuntu(UID=1000) 执行 sudo cat /etc/shadow

时间线：
                          uid     euid     suid     fsuid
                          ────    ────     ────     ─────
启动 bash 进程时：        1000    1000     1000     1000
                         (你是谁)(权限判定)(回退用)(文件判定)

执行 sudo 时 (execve 一个 setuid-root 程序)：
  sudo 进程：             1000    0        0        0
                          ↑       ↑        ↑        ↑
```

```
                          真实身份   当前权力   保存获授身份   文件访问权力
                          没变       变成root   root可回切     也变root

sudo 内部设置 UID 后 exec 目标命令（如 cat）：
  cat 进程:               0       0        0        0
                          全部是 root
                           (sudo 在 exec 前调用 setresuid(0,0,0)
                           将 uid/euid/suid/fsuid 全部设为 0)
```

字段	语义	何时设置	作用
uid (Real)	你真正是谁	login 时确定	标识身份，一般不变
euid (Effective)	你现在以谁的身份行事	`execve()` / `set*uid()` 等规则更新	很多权限检查最终都会看这里或由它派生
suid (Saved)	保留一份可回切的有效身份	`execve(setuid)` / `set*uid()` 规则更新	允许在许可范围内切回
fsuid (FS)	文件操作以谁的身份	通常跟 euid 联动	VFS 文件权限判定主要看它

请注意： 这里的 suid 和我们之前理解的 SUID 提权不是一个概念，这里的 suid 只是一个字段。

> 上述案例中为什么执行 sudo 时（execve 一个 setuid-root 程序）时 suid 改变了呢？ 它不是保留一份可回切的有效身份吗？

因为 suid 保存的不是"旧身份"，而是**新获授的身份**。

内核 execve 路径中的三步操作如下：

```
1. prepare_exec_creds()
   suid = fsuid = euid = 1000     ← 先同步到当前 euid（此时还是 1000）

2. bprm_fill_uid()
   euid = 0                        ← SUID 位生效，euid 改为文件属主（root）

3. cap_bprm_creds_from_file()
   suid = fsuid = euid = 0         ← 再次同步到新的 euid（已是 0）
```

第 3 步把 suid 同步到的是新 **euid（0）**，而不是旧 euid（1000）。

这是有意为之的设计：suid 的语义是"保存这份通过 SUID 机制获授的身份，以便后续回切"。对于 SUID-root 程序，获授的身份就是 root（0），所以 suid=0。这样 SUID 程序才能在需要时临时降权、再通过 suid 恢复：

```
sudo 进程初始状态：uid=1000, euid=0, suid=0
                       ↑真实身份     ↑当前权力     ↑保存的获授身份

临时降权：seteuid(1000)
→ uid=1000, euid=1000, suid=0     ← suid 仍保留 root 身份

恢复权力：seteuid(0)（内核检查 suid，允许回切）
→ uid=1000, euid=0, suid=0        ← 成功回到 root
```

如果 suid 保存的是旧值 1000，那 SUID 程序降权后就永远无法恢复了，SUID 机制就失去了意义。

2. 四种 GID 各自的角色

四种 GID 的机制与四种 UID 完全对称，规则一一对应：

GID 字段	对应的 UID 字段	语义	何时设置	作用
gid (Real)	uid	你真正属于哪个组	login 时确定	标识组身份，一般不变
egid (Effective)	euid	你现在以哪个组的身份行事	`execve()` / `set*gid()` 等规则更新	组权限判定的主要依据
sgid (Saved)	suid	保留一份可回切的有效组身份	`execve(sgid)` / `set*gid()` 规则更新	允许在许可范围内切回
fsgid (FS)	fsuid	文件操作以哪个组的身份	通常跟 egid 联动	VFS 文件组权限判定主要看它

还是用之前的场景，把 GID 也加上：

```
用户 ubuntu(UID=1000, GID=1000) 执行 sudo cat /etc/shadow

时间线（同时展示 UID 和 GID）：
              uid  euid  suid  fsuid | gid  egid  sgid  fsgid
              ────  ────  ────  ───── | ────  ────  ────  ─────
bash 进程:    1000 1000  1000  1000  | 1000 1000  1000  1000

execve sudo   1000   0     0     0  | 1000 1000  1000  1000
 (SUID 生效)  ↑ UID系列变了           ↑ GID 系列没变!
              SUID 位只影响 UID      sudo 没有 SGID 位

sudo 内部:
setresgid(0,0,0)                     |    0    0     0     0
                                         ↑ 此时 GID 才变
setresuid(0,0,0)               0    0     0     0  |

exec cat:      0    0     0     0  |  0    0     0     0
```

关键区别：UID 和 GID 变化的触发机制不同。

SUID 位只影响 UID 系列（`bprm_fill_uid()` 检测到 SUID 位后只修改 `euid`）。要让 GID 系列也变，需要文件同时设置 **SGID** 位，或者由程序代码主动调用 `setresgid()` 之类的函数。上面的案例中，sudo 没有 SGID 位，所以 GID 的变化完全是 sudo 自己代码里调用 `setresgid(0,0,0)` 主动完成的。

这也意味着：如果一个 SUID-root 程序只依赖 SUID 位提权、忘记处理 GID，那它的 gid/egid/sgid/fsgid 都还停留在原用户的组身份上，某些基于组权限的操作可能会出问题。

3. group_info（补充组列表）

前面介绍了四种 GID，它们存储的是进程的**主组身份**。但一个用户通常不只属于一个组——ubuntu 用户可能同时在 sudo 组、docker 组、adm 组里。这些**额外的组成员关系**就存储在 `group_info` 字段中。

主组和补充组的分工

```
cred 中与组相关的字段：

主组（只有一个数字）：
  gid  = 1000    ← 你真正的主组
  egid = 1000    ← 有效主组
  sgid = 1000    ← 保存的主组
  fsgid = 1000   ← 文件操作用的主组

补充组（一个已排序的数组）：
  group_info → [4, 27, 998]
                 ↑   ↑    ↑
                 adm sudo docker
```

主组来自 `/etc/passwd` 的第 4 字段，通过 `setgid()` / `setresgid()` 设置到 `cred->gid/egid/sgid/fsgid`。

补充组来自 `/etc/group`（通过 `initgroups()` 查询所有包含该用户的组），通过 `setgroups()` 系统调用设置到 `cred->group_info`。

权限检查时两者都会被查

在下面 0x05 讲的 `acl_permission_check()` 中，判断进程是否属于文件的 group 时，内核调用 `in_group_p()`：

```
// kernel/groups.c
int in_group_p(kgid_t grp)
{
    const struct cred *cred = current_cred();

    if (cred->fsgid == grp)              // 先查主组
        return 1;

    return groups_search(cred->group_info, grp);  // 再查补充组
}
```

先拿 `fsgid`（跟随主 GID）比对，匹配就通过；不匹配再查 `group_info` 里的补充组列表。两者**分别检查、各自独立**。

用之前的 `/etc/shadow` 案例：

```
文件 group = shadow(42)

进程 fsgid = 1000 → 1000 != 42 → 主组不匹配
进程 group_info = [4, 27, 998] → 没有找到 42 → 补充组也不匹配
→ 落入 other 位判断
```

补充组在登录时一次性加载

`group_info` 不是实时查 `/etc/group` 的。它在登录时由 `initgroups()` 一次性加载，钉到 `cred` 上，之后 fork 出的子进程全部继承。

这意味着：如果管理员在用户登录后把用户从 docker 组移除，现有会话的 `cred->group_info` 不会变化，一般要到新会话建立后才会生效。

一句话总结

> `gid/egid/sgid/fsgid` 存储主组（一个数字），`group_info` 存储补充组（一个已排序数组）。权限检查时两者都会被查，缺一不可。补充组在登录时一次性加载，不会在每次权限判定时回查组数据库。

4. usage（引用计数）

`usage` 是 `atomic_long_t` 类型，记录有多少个指针正在引用这个 cred 对象。它回答的问题是：**这个 cred 对象现在被几个 task_struct 的 `real_cred` / `cred` 指针指着？没有人指了就可以释放。**

基本规则

`task_struct` 中有两个 cred 指针（`real_cred` 和 `cred`），每多一个指针指向某个 cred 对象，`usage` 就 +1；少一个指针指向它，`usage` 就 -1。当 `usage` 降到 0 时，这个 cred 对象被释放回 slab 分配器。

场景一：普通进程（fork）

```
bash (cred_A, usage=2)
  ↑ real_cred 持有 1 份引用
  ↑ cred      持有 1 份引用
  所以 usage = 2

fork() 创建子进程 → prepare_creds()
  从 slab 分配全新的 cred_B，memcpy 拷贝 cred_A 的内容
  cred_B 是独立对象，usage = 2（子进程的 real_cred 和 cred 各持一份）
  cred_A 的 usage 不变，仍然是 2

结果：
  bash     → cred_A (usage=2)
  子进程   → cred_B (usage=2)  ← 独立副本，互不影响
```

场景二：线程（CLONE_THREAD，共享 cred）

```
主线程 (cred_A, usage=2)
  ↑ real_cred 持有 1 份
  ↑ cred      持有 1 份

clone(CLONE_THREAD) → copy_creds()
  不创建新 cred 对象！
  get_cred_many(cred_A, 2)  → usage += 2（新线程的 real_cred 和 cred 也指向 cred_A）

结果：
  主线程 → cred_A (usage=4)  ← 同一个对象
  新线程 → cred_A (usage=4)  ← real_cred +1, cred +1，共增加 2
```

50 个线程共享同一个 cred 对象时，`usage = 2 × 50 = 100`。

场景三：修改 cred（COW 机制）

```
三个线程共享 cred_A (usage=6)：
  线程1 → cred_A
  线程2 → cred_A
  线程3 → cred_A
```

```
线程3 调用 setresuid() 修改自己的 cred:
  1. prepare_creds()
     分配新的 cred_B, memcpy 拷贝 cred_A
     cred_B 的 usage = 2 (线程3 的 real_cred 和 cred 将各持一份)

  2. 修改 cred_B 的字段

  3. commit_creds(cred_B)
     线程3 的 real_cred 和 cred 从 cred_A 切换到 cred_B
     put_cred_many(cred_A, 2)  → cred_A 的 usage 从 6 减到 4

结果:
  线程1 → cred_A (usage=4)  ← 不受影响
  线程2 → cred_A (usage=4)  ← 不受影响
  线程3 → cred_B (usage=2)  ← 独立对象, 修改只影响自己
```

这就是 COW（Copy-On-Write）：要修改时先拷贝一份，改新的那份，旧的留给其他共享者继续用。`usage` 是这套机制能正常运转的前提——它确保一个 cred 对象不会在使用途中被意外释放。

一句话总结

> `usage` 是 cred 对象的生命线：每个指向它的指针算一份引用，引用归零时释放。fork 创建独立副本，线程共享同一个对象，修改时拷贝新对象——这些行为的背后都是 `usage` 在管理谁还在用这个 cred。

5. security（LSM 安全上下文）

Linux 除了传统的文件权限位（owner/group/other）和 capabilities 之外，还有一种更细粒度的安全机制：**LSM（Linux Security Modules）**。常见的 LSM 实现有 SELinux、AppArmor、Smack 等。

`security` 字段就是 LSM 模块在 cred 中存储自己安全上下文的地方。

它不是一个简单的指针

`security` 声明为 `void *`，但它指向的不是一个固定结构体，而是一块**按 LSM 模块需求拼接出来的内存区域（blob）**。系统上启用了哪些 LSM 模块，这块 blob 里就有哪些模块的数据：

```
cred->security 指向的内存:

┌──────────────────────────────┐
│ SELinux 的部分               │  ← struct cred_security_struct
│   sid = 1001 (当前进程的安全ID) │     (osid/sid/exec_sid/create_sid/...)
│   ...                        │
├──────────────────────────────┤
│ AppArmor 的部分              │  ← struct aa_label *
│   label → "unconfined"       │     (指向一个安全标签)
│   ...                        │
├──────────────────────────────┤
│ Smack 的部分 (如果启用)       │  ← struct task_smack
│   smk_task → "User"          │     (smk_task/smk_forked/smk_transmuted/...)
│   ...                        │
└──────────────────────────────┘

每个模块通过 cred->security + 自身偏移量 来找到自己的区域
```

不同 LSM 模块存储的内容不同：

LSM 模块	存储的结构体	核心内容
SELinux	`struct cred_security_struct`	SID（Security ID），进程的安全上下文标签
AppArmor	`struct aa_label *`	安全标签，表示进程的 confinement 策略
Smack	`struct task_smack`	进程标签、fork 标签、转换标签、访问规则

它如何参与权限检查

以 SELinux 为例，当进程调用 `open("/etc/shadow")` 时，内核除了检查传统的文件权限位（"你是不是文件的 owner？你的组有没有权限？"），**还会额外检查 LSM 规则**：

```
进程调用 open("/etc/shadow")

1. 传统权限检查：文件的 owner/group/other 权限位 + capability
   → 通过（假设进程有足够权限）

2. LSM 检查（SELinux）：
   → 从 cred->security 读取当前进程的 SID（安全标签）
   → 从 inode 自身的 LSM 数据读取文件的安全标签
   → 判定：这个进程标签能否访问这个文件标签？
   → 允许 or 拒绝
```

关键点：**即使传统权限位全部放行，LSM 仍然可以拒绝**。比如在启用 SELinux 的系统上，即使你是 root、文件权限是 777，SELinux 策略仍然可能因为进程标签和文件标签不匹配而拒绝访问。这就是 `security` 字段在权限体系中的角色——它在传统权限之上叠加了一层额外的安全控制。

它随 cred 一起 COW

`security` 字段和其他字段一样遵循 COW 机制。`prepare_creds()` 拷贝 cred 时，LSM 框架会调 `security_prepare_creds()` 为新 cred 的 `security` 分配独立的 blob 并拷贝内容。修改时只影响新 cred，不影响共享旧 cred 的其他线程。

一句话总结

> `security` 是 LSM 模块存储安全上下文的地方，它在传统的文件权限之上叠加了一层额外的安全控制——即使文件权限位允许访问，LSM 仍然可以基于 `cred->security` 中的标签拒绝它。

6. COW——修改 cred 时拷贝新对象

前面在介绍 `usage` 时已经通过场景展示了 COW 的效果，这里从机制层面做一个总结。

内核绝不原地修改 cred 的字段值（uid、gid 等），而是拷贝一份再改。核心函数只有四个：

函数	作用
`get_cred(cred)`	refcount++，有人要持有它
`put_cred(cred)`	refcount--，refcount==0 时释放
`prepare_creds()`	从 slab 分配新 cred，拷贝当前 cred 的内容
`commit_creds(new)`	将 task_struct 的 cred 指针替换为新 cred

以一个多线程进程中某个线程提权为例：

```
提权前，三个线程共享 cred_A：
  主线程(TID=1000)  → cred_A (uid=1000, euid=1000)
  工作线程1(TID=1001) → cred_A            ← 共享同一个
  工作线程2(TID=1002) → cred_A            ← 共享同一个

某线程调用 setresuid() 提权时：
  1. prepare_creds() → 新分配 cred_B，memcpy 拷贝 cred_A 的内容
  2. 修改 cred_B：
     cred_B->euid = 0;
     cred_B->fsuid = 0;
  3. commit_creds(cred_B)：
     该线程的 task_struct 的 real_cred 和 cred 指针替换为 cred_B
     put_cred_many(cred_A, 2) → 释放该线程对旧 cred 的引用

此时内存中的状态：
  主线程(TID=1000)  → cred_A (uid=1000, euid=1000) ← 不受影响
  工作线程1(TID=1001) → cred_A                      ← 不受影响
  提权线程(TID=1002)  → cred_B (uid=1000, euid=0)     ← 独立对象
```

线程之间共享 cred 是常态。一个多线程进程创建了 50 个工作线程，内核内存中只有一个 `cred` 对象被 50 个线程同时引用。而常规 fork 创建的子进程每次都会获得独立的 cred 副本。

一句话总结： cred 的修改永远遵循 prepare → modify → commit 三步模式，不会原地修改字段值。这保证了多线程共享 cred 时，一个线程的权限变化不会影响其他线程。

7. 小结

经过上述介绍，大家应该已经能够初步理解内核中进程，尤其是进程中的用户凭证信息在内核中的表示方式和存储格式了。

0x05 内核权限校验

前面我们详细了解了 `cred` 结构体中各个字段的含义，那这些字段在实际的权限校验中是如何被使用的呢？

这一节我们追踪一个具体的场景：普通用户执行 `cat /etc/shadow` 时被拒绝，内核到底做了什么？

1. 整体流程

当 `cat` 调用 `open("/etc/shadow")` 时，最终会进入内核的 `inode_permission()` 函数。这个函数按固定顺序串行执行多层检查，**任意一层拒绝则整体拒绝**：

```
cat 调用 open("/etc/shadow", O_RDONLY)
  ↓
inode_permission()
  │
  ├─ 第 1 层: sb_permission()
  │    文件系统级别的限制（如只读挂载直接拒绝写入）
  │
  ├─ 第 2 层: inode 属性检查
  │    不可变文件（immutable）拒绝写入
  │    ID 映射未映射的文件拒绝写入
  │
  ├─ 第 3 层: generic_permission()  ← DAC 主逻辑（核心）
  │    ├─ acl_permission_check(): 根据 fsuid/fsgid/group_info
  │    │   与 inode 的 owner/group/other 模式位做匹配
  │    │
  │    └─ DAC 不通过时，看 capability 能否绕过?
  │       CAP_DAC_READ_SEARCH: 绕过读限制
  │       CAP_DAC_OVERRIDE: 绕过读写执行限制
  │
  ├─ 第 4 层: devcgroup_inode_permission()
  │    设备 cgroup 限制（仅对字符/块设备节点生效）
  │
  └─ 第 5 层: security_inode_permission()
       LSM 钩子（SELinux / AppArmor / Smack 等）
```

下面逐层解析。

2. 第 1 层：文件系统级检查

```c
// fs/namei.c（简化）
static int sb_permission(struct super_block *sb, struct inode *inode, int mask)
{
    if (mask & MAY_WRITE) {
        // 只读挂载的文件系统上，普通文件/目录/符号链接不允许写入
        if (sb_rdonly(sb) && (S_ISREG(mode) || S_ISDIR(mode) || S_ISLNK(mode)))
            return -EROFS;
    }
    return 0;
}
```

这一层很粗糙，只看一件事：**这个文件系统是不是只读挂载的？** 如果是，写操作直接拒绝。对于我们的 `cat /etc/shadow`（读操作），这一层直接放行。

3. 第 2 层：inode 属性检查

```c
// fs/namei.c inode_permission() 中（简化）
if (mask & MAY_WRITE) {
    if (unlikely(IS_IMMUTABLE(inode)))
        return -EPERM;          // 不可变文件，谁都不能写
    if (unlikely(HAS_UNMAPPED_ID(idmap, inode)))
        return -EACCES;         // ID 映射有问题，拒绝写入
}
```

这一层检查 inode 自身的属性。比如文件被设置了 immutable 标志（`chattr +i`），那就直接拒绝写入。同样，对于我们的读操作，这一层也直接放行。

4. 第 3 层：DAC 主逻辑（核心）

这是最重要的一层，也是每次都会执行的核心权限检查。它分两步走：

第一步：基础 DAC / ACL 检查（`acl_permission_check`）

```
// fs/namei.c (简化)
static int acl_permission_check(struct mnt_idmap *idmap,
                                struct inode *inode, int mask)
{
    // 第一步：看 fsuid 是不是文件的 owner
    if (likely(vfsuid_eq_kuid(vfsuid, current_fsuid()))) {
        // 是 owner → 用 owner 的权限位判断 (mode >> 6)
        mask &= 7;
        mode >>= 6;
        return (mask & ~mode) ? -EACCES : 0;
    }

    // 第二步：不是 owner → 检查 POSIX ACL (如果文件有 ACL 规则)
    if (IS_POSIXACL(inode) && (mode & S_IRWXG)) {
        int error = check_acl(idmap, inode, mask);
        if (error != -EAGAIN)      // ACL 有明确结论 (允许或拒绝)
            return error;
        // -EAGAIN 表示 ACL 没有匹配到，继续往下走
    }

    // 第三步：检查 group/other 权限位
    // 看 fsgid 或 group_info 中的组有没有匹配 inode 的 group
    if (vfsgid_in_group_p(vfsgid))
        mode >>= 3;   // 匹配到组 → 用 group 权限位

    // 都没匹配 → 用 other 权限位 (mode 的低 3 位)
    return (mask & ~mode) ? -EACCES : 0;
}
```

用 `/etc/shadow` 的例子走一遍：

```
/etc/shadow 的 inode 信息:
  owner = root (UID=0)
  group = shadow (GID=42)
  mode  = 0640 → owner 可读写, group 可读, other 无权限
  无 POSIX ACL 规则

cat 进程的 cred:
  fsuid = 1000 (ubuntu)
  fsgid = 1000 (ubuntu)
  group_info = [27(sudo), 998(docker), 4(adm)]

acl_permission_check 三步走:

第一步: fsuid(1000) == inode owner(0)?  → 不是, 跳过

第二步: IS_POSIXACL(inode)?  → 否 (/etc/shadow 没有 ACL 规则), 跳过
```

```
第三步：group/other 权限位判断
  mask = MAY_READ = 4 = 100 (想读)
  vfsgid_in_group_p(42)? → fsgid=1000 不是 42，group_info 里也没有 42
  → 不右移，用 other 的低 3 位

  mode 低 3 位 = 000 (other 无权限)
  mask & ~mode = 100 & ~000 = 100 & 111 = 100 ≠ 0
  → 有请求但不被允许的权限
  → 返回 -EACCES
```

第二步：**capability 能否绕过？**

如果 DAC 检查不通过（`acl_permission_check` 返回 `-EACCES`），内核还会看进程有没有特殊的 capability 可以强行通过。这里目录和普通文件的规则不同：

```
// fs/namei.c generic_permission() (简化)
ret = acl_permission_check(idmap, inode, mask);
if (ret != -EACCES)
    return ret;    // 通过或非权限错误，直接返回

// ── 目录的 capability 绕过规则 ──
if (S_ISDIR(inode->i_mode)) {
    // 目录的非写操作 → CAP_DAC_READ_SEARCH 可绕过
    if (!(mask & MAY_WRITE))
        if (capable_wrt_inode_uidgid(idmap, inode, CAP_DAC_READ_SEARCH))
            return 0;
    // 目录的任何操作 → CAP_DAC_OVERRIDE 可绕过
    if (capable_wrt_inode_uidgid(idmap, inode, CAP_DAC_OVERRIDE))
        return 0;
    return -EACCES;  // 目录在这里就结束了
}

// ── 普通文件的 capability 绕过规则 ──
mask &= MAY_READ | MAY_WRITE | MAY_EXEC;  // 过滤掉额外标志位，只保留 rwx

// 纯读操作 → CAP_DAC_READ_SEARCH 可绕过
if (mask == MAY_READ)
    if (capable_wrt_inode_uidgid(idmap, inode, CAP_DAC_READ_SEARCH))
        return 0;

// 非执行操作 或 文件本身有执行位 → CAP_DAC_OVERRIDE 可绕过
if (!(mask & MAY_EXEC) || (inode->i_mode & S_IXUGO))
    if (capable_wrt_inode_uidgid(idmap, inode, CAP_DAC_OVERRIDE))
        return 0;

return -EACCES;
```

对于普通用户的 cat 进程：

- 没有 `CAP_DAC_READ_SEARCH` → 不行
- 没有 `CAP_DAC_OVERRIDE` → 不行
- 最终返回 `-EACCES`

这就是为什么 root 几乎能访问任何文件——在这里讨论的 `generic_permission()` / capability 绕过这一步里，并不是直接写了一个“`uid=0` 就放行”的特殊分支，而是 `uid=0` 的进程通常会按 capability 保留规则持有 `CAP_DAC_OVERRIDE` 等能力，因此可以在这一步直接绕过 DAC 限制。需要注意的是，这并不意味着内核在所有路径里都完全不会对 `uid=0` 做特殊处理；例如后面会提到的 `set*uid()` 相关 capability 整理逻辑，就会显式考虑 `uid=0` 的情况。

5. 第 4 层：设备 cgroup 检查

```
retval = devcgroup_inode_permission(inode, mask);
```

这一层只对字符设备（如 `/dev/null`）和块设备（如 `/dev/sda`）生效。对于普通文件（如 `/etc/shadow`），直接放行。这一层的作用是限制容器中的进程能访问哪些设备。

6. 第 5 层：LSM 检查

```
return security_inode_permission(inode, mask);
```

这就是上一节介绍的 `security` 字段发挥作用的地方。如果系统启用了 SELinux/AppArmor/Smack，它们会在这里做额外的安全判定。即使前面四层全部放行，这一层仍然可以拒绝。

7. 完整走一遍

把 `cat /etc/shadow` 的完整判定过程串起来：

```
cat (uid=1000, euid=1000, fsuid=1000)
  group_info = [27(sudo), 998(docker), 4(adm)]
  cap_effective = {} (没有任何 capability)

open("/etc/shadow")
  ↓
第 1 层 sb_permission()
  → 读操作，文件系统不是只读的 → 通过
  ↓
第 2 层 inode 属性
  → 读操作，不是 immutable → 通过
  ↓
第 3 层 generic_permission()
  acl_permission_check():
    fsuid=1000 ≠ owner=0           → 不是 owner
    group_info 里没有 GID=42       → 不是 group
    落入 other 位 → mode = 0       → 无权限
    → 返回 -EACCES

  capability 绕过检查:
    CAP_DAC_READ_SEARCH?           → 没有
    CAP_DAC_OVERRIDE?              → 没有
    → 返回 -EACCES
  ↓
第 3 层拒绝，直接返回 -EACCES
（第 4、5 层不会执行）

cat 收到 EACCES → 输出 "Permission denied"
```

现在换成 `sudo cat /etc/shadow`（cat 以 root 运行）：

```
cat (uid=0, euid=0, fsuid=0)
  cap_effective = {全部 capability}

第 3 层 generic_permission()
  acl_permission_check():
    fsuid=0 == owner=0              → 是 owner!
    owner 位: mode >> 6 = 6 (rw-)   → 有读权限
    → 返回 0 (通过)
```

root 甚至不需要走到 capability 绕过那一步——它的 fsuid=0 直接匹配了文件 owner，owner 位本身就有读写权限。

8. 一个常见误解的纠正

误解	实际机制
"root 直接跳过所有检查"	root 通过持有 capability 绕过 DAC，LSM 层仍可能拒绝
"拿 fsuid 和文件属主比一下就行"	还要查 group_info、ACL、capability、挂载属性、LSM
"每次 open 都会读 /etc/passwd"	全部基于 cred + inode 的内存数据，不查任何文本文件

内核权限判定本身不会去查 `/etc/passwd`、`/etc/group`、`/etc/shadow` 这种文本文件。 判定的全部依据都在内核内存中：当前进程的 `cred` 和文件的 `inode` 元数据。

0x06 uid 和 gid 是哪里来的？

前面说了内核只认 UID/GID 数字，那这些数字是从哪里来的？进程启动时 cred 里的 uid、gid 是怎么确定的？

答案是一条很长的链路，从硬盘上的文本文件一路到内核内存中的 cred 结构体：

```
/etc/passwd (存储 UID/GID)
    ↓
getpwnam() (glibc 通过 NSS 读取)
    ↓
struct passwd { pw_uid, pw_gid } (返回给调用者)
    ↓
sshd/login/sudo (登录程序拿到 UID/GID)
    ↓
setresuid() / setresgid() / setgroups() (系统调用)
    ↓
cred 结构体的 uid/euid/suid/fsuid/gid/egid/sgid/fsgid/group_info
```

下面逐环节拆解。

1. 存储层：/etc/passwd

用户的 UID 和 GID 存储在 `/etc/passwd` 中，每行一条记录，格式如下：

```
root:x:0:0:root:/root:/bin/bash
ubuntu:x:1000:1000:Ubuntu:/home/ubuntu:/bin/bash
```

字段含义（冒号分隔）：

字段位置	含义	示例
第 1 个	用户名	`ubuntu`
第 2 个	密码占位符（真实密码在 /etc/shadow）	`x`
第 3 个	**UID（用户 ID）**	**1000**
第 4 个	**GID（主组 ID）**	**1000**
第 5 个	用户描述（GECOS）	`Ubuntu`
第 6 个	家目录	`/home/ubuntu`
第 7 个	登录 shell	`/bin/bash`

这就是 UID/GID 的源头——一个文本文件里的数字。但内核不会直接读这个文件，中间还有好几层。

2. 读取层：getpwnam() 和 NSS

登录程序（如 sshd）需要根据用户名查出对应的 UID/GID，用的就是 C 标准库的 `getpwnam()` 函数：

```
// glibc 标准函数
struct passwd *getpwnam(const char *name);

// 返回的结构体
struct passwd {
    char   *pw_name;   // 用户名
    char   *pw_passwd; // 密码占位符
    uid_t   pw_uid;    // ← UID 在这里
    gid_t   pw_gid;    // ← GID 在这里
    char   *pw_gecos;  // 描述
    char   *pw_dir;    // 家目录
    char   *pw_shell;  // shell
};
```

`getpwnam()` 内部并不是直接读 `/etc/passwd`，而是通过 **NSS（Name Service Switch）** 框架查询。NSS 的配置在 `/etc/nsswitch.conf`：

```
# /etc/nsswitch.conf 示例
# 注意：不同系统、不同用途的机器配置差异很大，这只是其中一种

# Ubuntu 桌面/服务器常见配置：
passwd:     files systemd
group:      files systemd
shadow:     files systemd

# 接入了企业 AD/LDAP 的机器可能长这样：
# passwd:     files sss
# group:      files sss
# shadow:     files sss
```

每行配置的含义是：**按从左到右的顺序依次查询，查到就停**。上面的例子中 `passwd: files sss` 表示先查本地文件（`files` = `/etc/passwd`），查不到再通过 SSSD 查远程目录服务。

常见的 NSS 来源：

NSS 来源	说明	典型场景
`files`	读本地 `/etc/passwd` 和 `/etc/group`	个人机器、小团队服务器
`systemd`	systemd 提供的动态用户解析（systemd-resolved、动态用户等）	现代 Ubuntu/CentOS 系统的默认配置
`sss`	通过 SSSD 查询（支持 AD、LDAP、IPA 等）	企业环境中集中管理账号
`ldap`	直接从 LDAP 服务器查询	传统 LDAP 部署
`winbind`	通过 Winbind 查询 Windows AD 域	Windows 域集成
`nis`	Network Information Service（旧称 Yellow Pages）	老式 UNIX 网络环境
`compat`	兼容模式，支持 `/etc/passwd` 中的 `+`/`-` 语法（配合 NIS 使用）	需要向后兼容旧 NIS 配置的系统
`db`	从 Berkeley DB 数据库文件查询	需要快速查表的大规模本地用户系统

注意 `passwd` 和 `group` 两行都很重要：`passwd` 行决定 UID/GID 从哪来，`group` 行决定补充组列表从哪来（后面讲 `initgroups()` 时会用到）。

无论哪种来源，对调用者来说接口都一样：`getpwnam("ubuntu")` 返回一个 `struct passwd`，里面有 `pw_uid=1000` 和 `pw_gid=1000`。

3. 登录程序层：sshd 如何拿到 UID/GID

以 OpenSSH 的 sshd 为例，认证通过后，sshd 调用 `getpwnam()` 查询用户信息（auth.c）：

```c
// openssh-portable/auth.c
struct passwd *
getpwnamallow(struct ssh *ssh, const char *user)
{
    // ... 安全检查 ...

    pw = getpwnam(user);  // ← 这里查出 struct passwd，包含 UID/GID

    // ... 权限验证 ...
    return pw;
}
```

此后 `pw->pw_uid` 和 `pw->pw_gid` 就在 sshd 进程的用户空间内存中了。接下来 sshd 要做的，就是把这些数字**通过系统调用交给内核**，让内核写入 cred。

4. 设置层：从用户空间到内核 cred

sshd 拿到 UID/GID 后，按顺序调用四个关键函数（基于 OpenSSH 源码 uidswap.c 和 session.c）：

第一步：**setgid() — 设置主 GID**

```c
// openssh-portable/session.c
setgid(pw->pw_gid);   // 例如 setgid(1000)
```

内核收到后（`__sys_setgid`），创建新 cred，设置 `gid = egid = sgid = fsgid = 1000`，commit。

第二步：**initgroups() — 设置补充组**

```c
// openssh-portable/session.c
initgroups(pw->pw_name, pw->pw_gid);  // 例如 initgroups("ubuntu", 1000)
```

`initgroups()` 是 glibc 函数，它内部会：

1. 通过 NSS 查询 `/etc/group`（或 LDAP），找出 ubuntu 所属的所有组
2. 调用 `setgroups()` 系统调用，把组列表交给内核

内核的 `setgroups()` 实现（kernel/groups.c）：

```c
SYSCALL_DEFINE2(setgroups, int, gidsetsize, gid_t __user *, grouplist)
{
    group_info = groups_alloc(gidsetsize);     // 分配 group_info 结构
    groups_from_user(group_info, grouplist);   // 从用户空间拷贝组列表
    groups_sort(group_info);                   // 排序（供后续二分查找用）
    set_current_groups(group_info);            // 写入 cred->group_info
}
```

最终 `cred->group_info` 里就有了 `[27(sudo), 998(docker), 4(adm), ...]`。

第三步：**permanently_set_uid() — 永久切换 UID**

```c
// openssh-portable/uidswap.c
void permanently_set_uid(struct passwd *pw)
{
    setresgid(pw->pw_gid, pw->pw_gid, pw->pw_gid);  // 再确认一次 GID
    initgroups(pw->pw_name, pw->pw_gid);              // 再确认一次补充组
    setresuid(pw->pw_uid, pw->pw_uid, pw->pw_uid);   // 切换 UID
}
```

`setresuid(1000, 1000, 1000)` 的内核实现（kernel/sys.c）：

```c
long __sys_setresuid(uid_t ruid, uid_t euid, uid_t suid)
{
    new = prepare_creds();          // COW: 分配新 cred，拷贝当前内容

    if (ruid != (uid_t)-1)
        new->uid = kruid;           // real UID = 1000
    if (euid != (uid_t)-1)
        new->euid = keuid;          // effective UID = 1000
    new->fsuid = new->euid;         // fsuid 跟随 euid = 1000
    if (suid != (uid_t)-1)
        new->suid = ksuid;          // saved UID = 1000

    return commit_creds(new);       // 替换 task_struct 的 cred 指针
}
```

请注意：上面的代码是为了说明 `cred` 字段如何被更新而做的**简化示意**，省略了真实内核里非常关键的一步权限检查。实际源码会先判断新设置的 `ruid/euid/suid` 是否超出了当前进程已有的 `{uid, euid, suid}` 可切换范围；如果超出，则必须具备 `CAP_SETUID`，否则直接返回 `-EPERM`。也就是说，普通用户不能靠 `setresuid(0, 0, 0)` 直接把自己提权成 root。

`setresgid()` 同理，设置 `gid/egid/sgid/fsgid` 四个字段。

5. 完整走一遍

用 SSH 登录 ubuntu 用户为例，追踪 cred 对象在每一步的变化：

```
SSH 连接建立
  ↓
sshd 主进程 fork 出子进程
  子进程继承 root 的 cred_A:
    uid=0, euid=0, gid=0, group_info=[0(root)]
  ↓
PAM 认证通过，确认用户是 ubuntu
  ↓
sshd 调用 getpwnam("ubuntu")
  → glibc 通过 NSS 查询 /etc/passwd
  → 返回 struct passwd { pw_uid=1000, pw_gid=1000, ... }
  ↓
sshd 调用 setgid(1000)                    ← 必须 root 身份才能调
  → prepare_creds() 分配 cred_B，拷贝 cred_A
  → cred_B.gid = cred_B.egid = cred_B.sgid = cred_B.fsgid = 1000
  → commit_creds(cred_B)，释放 cred_A

  子进程现在持有 cred_B:
    uid=0, euid=0, gid=1000, group_info=[0(root)]
    ↑ UID 还是 root，GID 已经切到 ubuntu
```

```
  ↓
sshd 调用 initgroups("ubuntu", 1000)
  → glibc 通过 NSS 查询 /etc/group
  → 找到 ubuntu 所属的组: sudo(27), docker(998), adm(4)
  → 调用 setgroups() 系统调用
  → prepare_creds() 分配 cred_C, 拷贝 cred_B
  → cred_C.group_info = [4, 27, 998] (已排序)
  → commit_creds(cred_C), 释放 cred_B

  子进程现在持有 cred_C:
    uid=0, euid=0, gid=1000, group_info=[4,27,998]
    ↑ UID 还是 root, 但补充组已经加载好了
  ↓
sshd 调用 setresuid(1000, 1000, 1000)      ← drop root, 不可回退
  → prepare_creds() 分配 cred_D, 拷贝 cred_C
  → cred_D.uid = cred_D.euid = cred_D.suid = cred_D.fsuid = 1000
  → commit_creds(cred_D), 释放 cred_C

  子进程现在持有 cred_D:
    uid=1000, euid=1000, gid=1000, group_info=[4,27,998]
    ↑ 完全以 ubuntu 身份运行, 不再是 root
  ↓
sshd 调用 execve("/bin/bash")
  → prepare_exec_creds() 分配 cred_E, 拷贝 cred_D
  → bash 不是 SUID 程序, 不修改任何字段
  → commit_creds(cred_E), 释放 cred_D

  bash 持有 cred_E:
    uid=1000, euid=1000, gid=1000, group_info=[4,27,998]
    ↑ 内容与 cred_D 相同, 但是独立的新对象
  ↓
后续 bash fork 出的所有子进程都从这个 cred_E 继承
```

可以看到，从 cred_A 到 cred_E，每次修改都是 prepare → modify → commit 三步，每次都分配新对象、释放旧对象。这正是前面讲 COW 机制时的模式。

6. 关键认知

认知	说明
内核不读 /etc/passwd	内核只接受系统调用传入的数字（UID/GID），不关心这些数字从哪来
UID/GID 的来源可能是远程的	通过 NSS，UID/GID 可以来自 LDAP、AD、SSSD，不一定是本地文件
补充组也是一次性加载的	`initgroups()` 在登录时查询组数据库，结果钉到 `cred->group_info`，之后不再回查
cred 一旦设置就独立存在	登录程序设置完 cred 后，无论原来的 `/etc/passwd` 怎么改，现有 cred 不会自动更新

0x07 cred 在进程生命周期中的流转

前面讲了 cred 的结构、权限校验、UID/GID 的来源。这一节用具体场景追踪 cred 对象在进程生命周期中如何传递和变化。

我们已经在前一节看过 SSH 登录建立初始 cred 的完整过程（从 cred_A 到 cred_E）。这里看其他常见场景。

1. 普通命令执行（fork + exec）

最常见的路径——bash 执行 `ls`：

```
bash (cred_A: uid=1000, euid=1000)
  │
  ├─ fork()
  │    copy_creds() → prepare_creds()
  │    从 slab 分配新对象，memcpy 拷贝 cred_A 的内容
  │    子进程拿到独立的 cred_B（内容与 cred_A 相同，对象不同）
  │
  ├─ 子进程 execve("/bin/ls")
  │    prepare_exec_creds() → prepare_creds()
  │    又分配新对象 cred_C，memcpy 拷贝 cred_B 的内容
  │    ls 不是 SUID 程序 → bprm_fill_uid() 不修改任何字段
  │    cred_B 被释放（put_cred），ls 使用 cred_C
  │
  └─ 结果：
       bash → cred_A (uid=1000, euid=1000)  ← 不变
       ls   → cred_C (uid=1000, euid=1000)  ← 独立对象，内容相同
```

一次普通命令执行涉及**两次 cred 创建**：fork 时一次，exec 时又一次。中间的 cred_B 生命期极短。

2. SUID 程序执行（fork + exec SUID 文件）

执行 `/usr/bin/passwd`（属主 root，设置了 SUID 位）：

```
bash (cred_A: uid=1000, euid=1000)
  │
  ├─ fork()
  │    子进程获得独立的 cred_B (uid=1000, euid=1000)
  │
  ├─ 子进程 execve("/usr/bin/passwd")
  │    1. prepare_exec_creds()
  │       分配 cred_C，memcpy cred_B
  │       cred_C: uid=1000, euid=1000, suid=1000, fsuid=1000
  │
  │    2. bprm_fill_uid()
  │       检测到 SUID 位 → cred_C->euid = 0
  │       cred_C: uid=1000, euid=0, suid=1000, fsuid=1000
  │
  │    3. cap_bprm_creds_from_file()
  │       处理 capabilities、安全检查
  │       同步：cred_C->suid = cred_C->fsuid = cred_C->euid
  │       cred_C: uid=1000, euid=0, suid=0, fsuid=0
  │
  │    4. commit_creds(cred_C)
  │       替换进程的 cred 指针，释放 cred_B
  │
  └─ 结果：
       bash   → cred_A (uid=1000, euid=1000)  ← 不变
       passwd → cred_C (uid=1000, euid=0, suid=0, fsuid=0)  ← 提权完成
```

注意这里 cred_C 的三步变化：先 memcpy 拷贝（suid/fsuid 同步到当前 euid=1000），再由 SUID 位改 euid=0，最后 cap_bprm_creds_from_file 把 suid/fsuid 同步到新的 euid=0。这就是前面 0x04 中讲的"suid 保存的是新获授的身份"的具体体现。

3. sudo 执行命令（SUID + setresuid + exec）

`sudo cat /etc/shadow` 的完整流程：

```
bash (cred_A: uid=1000, euid=1000)
  │
  ├─ fork() + execve("/usr/bin/sudo")
  │    同场景二，sudo 获得：（sudo 本身也是一个 SUID 权限的程序）
  │    cred_B: uid=1000, euid=0, suid=0, fsuid=0
  │
  ├─ sudo 内部（此时 euid=0，有 root 权限）
  │    读取 /etc/sudoers，检查 ubuntu 是否有权限
  │    通过 PAM 重新验证用户身份
  │    确认可以执行 cat /etc/shadow
  │
  │    sudo 调用 setresgid(0, 0, 0)
  │    → prepare_creds() 分配 cred_C，gid 系列全部设为 0
  │    → commit_creds(cred_C)，释放 cred_B
  │
  │    sudo 调用 setresuid(0, 0, 0)
  │    → prepare_creds() 分配 cred_D
  │    → uid=0, euid=0, suid=0, fsuid=0（全部设为 root）
  │    → commit_creds(cred_D)，释放 cred_C
  │
  ├─ sudo execve("/bin/cat")
  │    prepare_exec_creds() 分配 cred_E
  │    cat 不是 SUID 程序 → 不修改
  │    cred_E: uid=0, euid=0, suid=0, fsuid=0
  │    释放 cred_D
  │
  └─ 结果：
       bash → cred_A (uid=1000, euid=1000)
       cat  → cred_E (uid=0, euid=0, suid=0, fsuid=0)  ← 完全以 root 运行
```

sudo 比直接执行 SUID 程序多了一步：它在获得 euid=0 之后，还主动调用 `setresuid(0,0,0)` 把 uid 也改成 0。这是因为 sudo 的设计目标是"以目标用户身份运行命令"，不是"以 SUID 方式运行"——所以它要把所有 UID 字段都设为目标用户的值。

请注意: sudo 本身就是一个具备 SUID 权限的程序

```
test@linuxenv:~$ ls -al /usr/bin/sudo
-rwsr-xr-x 1 root root 277936 Mar  2 12:56 /usr/bin/sudo
test@linuxenv:~$ ls -al /bin/cat
```

4. 创建线程（共享 cred）

多线程进程中创建线程：

```
主线程 (cred_A: uid=1000, euid=1000, usage=2)
  │        ↑ real_cred 和 cred 各持一份引用
  │
  ├─ clone(CLONE_THREAD)
  │    copy_creds() 检测到 CLONE_THREAD
  │    → get_cred_many(cred_A, 2)  ← 不创建新对象, refcount += 2
  │    新线程的 real_cred 和 cred 也指向 cred_A
  │
  └─ 结果:
       主线程   → cred_A (usage=4)  ← 同一个对象
       新线程   → cred_A (usage=4)  ↑ real_cred 和 cred 各 +1, 两个线程共 4
```

线程之间共享 cred 是常态。一个多线程进程创建了 50 个工作线程，内核内存中只有一个 `cred` 对象被 50 个线程同时引用。

这也意味着如果某个线程修改了自己的 cred（比如调用 `setresuid`），COW 机制会为该线程创建独立的新 cred，其他线程不受影响。

5. 总结对照

场景	fork	exec	cred 变化
普通命令 `ls`	新建副本	新建副本，内容不变	全部不变：uid=1000, euid=1000, suid=1000, fsuid=1000
SUID 程序 `passwd`	新建副本	新建副本 + bprm_fill_uid 修改 euid	uid 不变(1000)，euid/suid/fsuid: 1000 → 0
sudo 执行命令	新建副本	第一次 exec sudo：SUID 使 euid=0 → setresuid(0,0,0) 全设 root → 第二次 exec cat：普通继承	uid: 1000 → 0，其余已经为 0，最终全部为 0
创建线程	不创建新 cred，共享	—	不变，共享同一对象
SSH 登录（0x06 已详述）	—	setgid → initgroups → setresuid → exec bash	uid/euid/suid/fsuid: 0 → 1000，gid: 0 → 1000，补充组加载

核心规律： fork 和 exec 都会调用 `prepare_creds()` 创建新 cred 对象（线程除外）。cred 的修改永远遵循 prepare → modify → commit 三步模式，不会原地修改字段值。

0x08 提权的本质

在网络安全领域，"提权"（Privilege Escalation）是最常见的攻击目标之一。攻击者在获得初始访问后（例如拿到了一个低权限 shell），通常需要提升权限才能完成更有价值的操作。

前面七节的内容已经完全覆盖了理解提权所需的所有基础知识。这一节用 cred 的视角重新审视提权，你会发现所有提权手法归结到内核层面只有一件事。

1. 提权的唯一定义

提权 = 让进程拿到更强的 cred

"更强"可以是以下任何一种变化：

变化	效果	举例
`euid` 从非 0 变成 0	获得 root 身份，几乎可以做任何事	SUID 程序、sudo
`cap_effective` 获得额外 capability	绕过特定权限限制	File Capabilities
LSM 上下文从受限变为不受限	绕过 SELinux/AppArmor 约束	SELinux 策略配置错误
进入更高权限的 user_namespace	在容器中获得宿主范围的权限	容器逃逸

无论攻击者用什么手法、走什么路径，最终目标都是让某个进程的 cred 变成上述状态之一。理解了这一点，就能把所有提权手法归类到统一的框架下。

2. SUID 程序——内核替你改 cred

0x07 已经展示了 SUID 程序的正常工作流程：`execve()` 时 `bprm_fill_uid()` 检测到 SUID 位，自动将新进程的 `euid` 设为文件属主。这是内核设计的合法机制，`passwd`、`sudo`、`su` 都依赖它工作。

攻击者的思路是：**找到系统中那些属主为 root 且设置了 SUID 位的程序，利用其中的逻辑缺陷来执行任意代码。**

常见的发现方式：

```
# 查找系统中所有 SUID 程序
find / -perm -4000 -type f 2>/dev/null

# 典型输出：
/usr/bin/sudo
/usr/bin/passwd
/usr/bin/su
/usr/bin/newgrp
/usr/bin/gpasswd
/usr/bin/chsh
/usr/bin/chfn
/usr/local/bin/custom_backup  ← 自定义程序，可能有问题
```

假设 `/usr/local/bin/custom_backup` 是一个 SUID-root 程序，内部调用 `system("tar cf /tmp/backup.tar /some/path")`：

```
攻击流程：
  攻击者 shell (uid=1000, euid=1000)
    │
    ├─ execve("/usr/local/bin/custom_backup")
    │     内核 bprm_fill_uid() 检测到 SUID 位
    │     新 cred: uid=1000, euid=0, suid=0, fsuid=0
    │     ↑ euid 已经是 root!
    │
    ├─ custom_backup 内部调用 system("tar cf ...")
    │     system() 通过 /bin/sh -c 执行
    │     攻击者通过 PATH 环境变量控制 /bin/sh 指向自己的程序
    │     或者利用 tar 的参数注入
    │
```

```
    └─ 攻击者的代码以 euid=0 运行
        → 完全控制 cred，任意操作
```

cred 视角：内核在 execve 时帮你把 `euid` 改成了 0，攻击者只需要找到一种方式让这个 euid=0 的进程执行自己的代码。

GTFOBins (https://gtfobins.github.io/) 是一个公开的资源，收录了 Unix 系统中可被利用的合法程序及其 SUID/sudo 等利用方式。

3. sudo 配置不当——合法的提权通道

sudo 本身就是一个 SUID-root 程序（0x07 已详述），它的设计目标就是"让授权用户以其他用户身份执行命令"。如果 sudoers 配置不当，攻击者不需要任何漏洞就能提权。

常见的危险配置：

```
# /etc/sudoers 中的危险配置示例

# 危险：允许无密码执行任何命令
ubuntu ALL=(ALL) NOPASSWD: ALL

# 危险：允许以 root 执行可以逃逸的程序
ubuntu ALL=(root) NOPASSWD: /usr/bin/vim
# vim 中执行 :!bash 即可获得 root shell

# 危险：允许执行可以读写文件的程序
ubuntu ALL=(root) NOPASSWD: /usr/bin/find
# sudo find /etc/shadow -exec cat {} \;

# 危险：允许执行解释器
ubuntu ALL=(root) NOPASSWD: /usr/bin/python3
# sudo python3 -c 'import os; os.system("/bin/bash")'
```

cred 视角：sudo 通过 SUID 机制获得 `euid=0`，然后调用 `setresuid(0,0,0)` 将所有 UID 字段设为 0，再 exec 目标命令。整个过程都是"合法"的 cred 变更，内核不会阻止。攻击者做的只是让 sudo exec 了一个能逃逸到 shell 的程序。

4. Capabilities——比 SUID 更细粒度的提权

SUID 是"全有或全无"的提权——euid 一变成 0，进程就获得了 root 的全部权限。Linux 从 2.2 开始引入 Capabilities 机制，将 root 权限拆分成独立的"能力"，前面 0x04 已经介绍了 cred 中的 5 个 capability 集合。

对攻击者来说，Capabilities 同样可以成为提权路径。关键在于某些 capability 本身就足够危险：

Capability	提权潜力	原因
`CAP_SETUID`	**直接提权到 root**	可以调用 `setresuid(0,0,0)` 将自己变成 root
`CAP_SETGID`	**直接提权**	可以加入任何组，读取组权限保护的文件
`CAP_DAC_OVERRIDE`	**读取任意文件**	绕过所有文件权限检查（0x05 已详述）
`CAP_DAC_READ_SEARCH`	**读取任意文件**	绕过文件读权限检查
`CAP_SYS_ADMIN`	**接近 root**	被称为"the new root"，能做的事极多（mount、namespace、bpf...）
`CAP_SYS_PTRACE`	**进程注入**	可以读写其他进程的内存，注入 shellcode
`CAP_NET_RAW`	**网络嗅探**	可以监听网络流量，抓取明文凭证

以 `CAP_SETUID` 为例：

```
# 查找具有 file capabilities 的程序
getcap -r / 2>/dev/null

# 如果发现 python3 被赋予了 CAP_SETUID:
/usr/bin/python3 = cap_setuid+ep
```

```
攻击流程:
  攻击者 shell (uid=1000, euid=1000)
    │
    ├─ execve("/usr/bin/python3")
    │    cap_bprm_creds_from_file() 读取 xattr security.capability
    │    新 cred: cap_effective 包含 CAP_SETUID
    │
    ├─ python3 中执行:
    │    import os
    │    os.setresuid(0, 0, 0)   ← CAP_SETUID 允许这样做!
    │
    └─ cred 变为 uid=0, euid=0, suid=0, fsuid=0
         → 完全提权到 root
```

cred 视角：File Capabilities 在 execve 时通过 `cap_bprm_creds_from_file()` 写入新 cred 的 `cap_permitted` 和 `cap_effective`。进程获得这些能力后，可以主动调用 `setresuid()` 等系统调用来进一步修改自己的 cred。

File Capabilities 的生效细节

file capabilities 存储在文件的 extended attribute `security.capability` 中。`execve()` 时，内核通过 `cap_bprm_creds_from_file()`（`security/commoncap.c`）读取该 xattr 并计算新凭证。新进程的 `cap_permitted` 按公式计算：

```
pP' = (bounding_set & file_permitted) | (old_inheritable & file_inheritable)
```

而 `cap_effective` 是否等于 `cap_permitted`，取决于 xattr 中的 **effective 标志位**（`VFS_CAP_FLAGS_EFFECTIVE`，即 `magic_etc` 的 bit 0）：

- 标志为 1 → `cap_effective = cap_permitted`（文件指定的全部 permitted caps 立即可用）
- 标志为 0 → `cap_effective = cap_ambient`（仅保留 ambient caps）

以 `ping` 为例：它的 `security.capability` xattr 中 `cap_net_raw` 在 permitted 集合内，且 effective 标志为 1，所以 execve 后 `cap_effective` 和 `cap_permitted` 都包含 `CAP_NET_RAW`，ping 可以直接使用原始套接字。

SUID 与 File Capabilities 同时存在的交互规则

当一个文件同时设置了 SUID-root 和 file capabilities 时，内核的处理不是简单叠加（security/commoncap.c）：

- **euid 设置**：始终生效（`bprm_fill_uid()` 先于 caps 处理执行）
- **file capabilities**：始终会被读取并应用到 `cap_permitted`
- **root 全权限 caps**：对非 root 用户执行同时设有 SUID-root 和 file capabilities 的文件，内核**不叠加** root 的全权限 capabilities，只保留 file capabilities 指定的范围，并打印一次警告

这意味着管理员如果给一个 SUID-root 程序额外设置了 file capabilities，反而可能**收窄**其权限范围——一个安全上容易踩的坑。

5. 内核漏洞——直接篡改 cred

前面四种方法都是在内核设计的规则内修改 cred——利用 SUID 位、sudo 授权、file capabilities 这些合法机制。内核漏洞则完全不同：它绕过所有规则，直接在内核空间篡改 cred 对象。

这是最强大也最底层的提权方式——不需要任何 SUID 程序、不需要 sudo 权限、不需要 file capabilities，只需要一个内核漏洞。

经典手法经历了内核对抗的演进：

```
// Linux v6.2 之前的经典 getroot（已封堵）：
commit_creds(prepare_kernel_cred(NULL));
// prepare_kernel_cred(NULL) 曾会退化成复制 init_cred（全权 cred），
// 但 v6.2 起（commit 3d6f83df875c）传入 NULL 直接返回 NULL，不再奏效。

// 现代内核漏洞利用的替代手法（示意）：
// 方式一：以 init_task 为模板创建全权 cred
commit_creds(prepare_kernel_cred(&init_task));

// 方式二：直接覆写 current 的 cred 指针，指向预先构造好的 cred
//  （绕过 commit_creds 的安全检查）
```

```
攻击流程（概念性描述）：
  攻击者 shell (uid=1000, euid=1000)
    │
    ├─ 触发内核漏洞（如越界写、UAF 等）
    │     → 获得在内核态执行任意代码的能力
    │
    ├─ 在内核态执行：
    │     commit_creds(prepare_kernel_cred(&init_task));
    │     // 为当前进程创建全权 cred 并替换
    │
    └─ 攻击者的 shell 进程 cred 变为：
          uid=0, euid=0, suid=0, fsuid=0
          cap_effective = 全部 capability
          → 完全提权到 root
```

cred 视角：内核漏洞让攻击者获得了在内核态执行代码的能力。此时 `current` 指向攻击者控制的进程，攻击者直接调用 `commit_creds()` 或覆写 cred 指针，让进程的 cred 变成全权状态。之前 0x04 中讲的 COW、引用计数、slab 分配等机制在这里全部失效——因为攻击者的代码运行在内核态，拥有对所有内核内存的访问权。

6. 其他常见路径

除了上述四类主要手法，还有一些间接路径——它们的共同特点是：不需要修改 cred，只需要让一个已经持有高权限 cred 的进程执行攻击者的代码。

定时任务（Cron）

```
# 查看系统定时任务
cat /etc/crontab
ls -la /etc/cron.d/

# 如果某个 root 的定时任务执行了攻击者可写的脚本
# 那个脚本就会以 root 的 cred 运行
```

cred 视角：cron 守护进程以 root 身份运行，执行任务时 fork + exec，子进程继承 root 的 cred（uid=0, euid=0）。攻击者不需要修改 cred，只需要让一个已经持有 root cred 的进程执行自己的代码。

服务配置不当

```
# 某些服务以 root 运行，但有 Web 接口或配置文件可被低权限用户修改
# 例如：Web 应用以 root 运行，存在文件写入漏洞
# 攻击者写入 PHP/Python webshell
# Web 服务器进程已有 root cred，webshell 自然继承
```

cred 视角：与定时任务相同——进程已经持有了高权限 cred，攻击者只需要找到一种方式影响它的行为，让它执行攻击者的代码。

7. 小结

攻击手法	cred 如何变化	前提条件
SUID 程序利用	execve 时内核设 `euid=0`	找到可利用的 SUID-root 程序
sudo 配置不当	sudo 调用 `setresuid(0,0,0)`	sudoers 中有过于宽松的规则
Capabilities 利用	execve 时内核添加 `cap_effective` 位	找到有危险 capability（如 CAP_SETUID）的程序
内核漏洞	直接调用 `commit_creds()`	存在可利用的内核漏洞
定时任务/服务	无需改 cred，进程已有高权限 cred	找到可注入的 root 任务或服务

所有提权手法归结到同一底层模型：让当前进程拿到更强的 `cred`

0x09 总结

回到开头的问题：在用户登录、执行命令、提权的过程中，Linux 系统底层到底发生了什么？

答案的核心只有一个数据结构——`cred`。

- **用户是什么？** 内核不认用户名，只认数字。一个"用户"在内核中的全部体现，就是挂载在 `task_struct` 上的一个 `cred` 对象里的 UID/GID
- **权限怎么判？** 进程每次访问文件时，内核拿 `cred` 中的 fsuid/fsgid/group_info/capabilities 与 inode 的模式位做匹配，全程基于内核内存中的数据，不查任何文本文件
- **权限怎么变？** cred 的修改永远遵循 prepare → modify → commit 的 COW 模式。无论是 SUID 程序的 execve、sudo 的 setresuid，还是 SSH 登录时的身份切换，底层都是同一个机制
- **提权是什么？** 所有提权手法——SUID 利用、sudo 配置不当、Capabilities、内核漏洞、服务注入——归结到底都是让进程拿到一个更强的 `cred`

Linux 内核就是依靠这样一套系统实现了用户的管理，关于 PAM 认证等内容可以参考往期文章